



Parameter Efficient Fine-Tuning (PEFT)



Mohammad Javad Dousti

Sources:

* Matt Gormley slides for CMU's Generative AI course.

* V. Lialin, V. Deshpande, and A. Rumshisky, "Scaling Down to Scale Up: A Guide to Parameter-Efficient Fine-Tuning", ArXiv 2023.

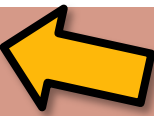
Few-shot Learning with LLMs

□ Suppose you have...

- a dataset $D = \{(x_i, y_i)\}_{i=1}^N$ and N is rather small (i.e. few-shot setting)
 - a very large (billions of parameters) pre-trained language model
- There are two ways to “learn”

Option A: Supervised fine-tuning

- **Definition:** fine-tune the LLM on the training data using...
 - a standard supervised objective
 - backpropagation to compute gradients
 - your favorite optimizer (e.g. Adam)
- **Pro:** fits into the standard ML recipe
- **Pro:** still works if N is large
- **Con:** backpropagation requires $\sim 3\times$ the memory and computation time as the forward computation
- **Con:** you might not have access to the model weights at all (e.g. because the model is proprietary)



In this section, we consider the question:

How can we do supervised finetuning of a very large foundation model more efficiently?

- **Con:** the prompt may be very long and Transformer LMs require $O(N^2)$ time/space where N = length of context
- **Pro:** no backpropagation required and only one pass through the training data
- **Pro:** does not require model weights, only API access

Fine-Tuning vs. In-Context Learning

- ❑ Why would we ever bother with fine-tuning if it's so inefficient?
- ❑ Because, even for very large LMs, fine-tuning often beats in-context learning

Method	MNLI-m (Val. Acc./%)	RTE (Val. Acc./%)
GPT-3 Few-Shot	40.6	69.0
GPT-3 Fine-Tuned	89.5	85.4

Question:

Why did fine-tuning of GPT-3 do so much better on these two tasks than few-shot learning?

Fine-Tuning vs. In-Context Learning

- ❑ Why would we ever bother with fine-tuning if it's so inefficient?
- ❑ Because, even for very large LMs, fine-tuning often beats in-context learning
- ❑ In a fair comparison of fine-tuning (FT) and in-context learning (ICL), we find that FT outperforms ICL for most model sizes

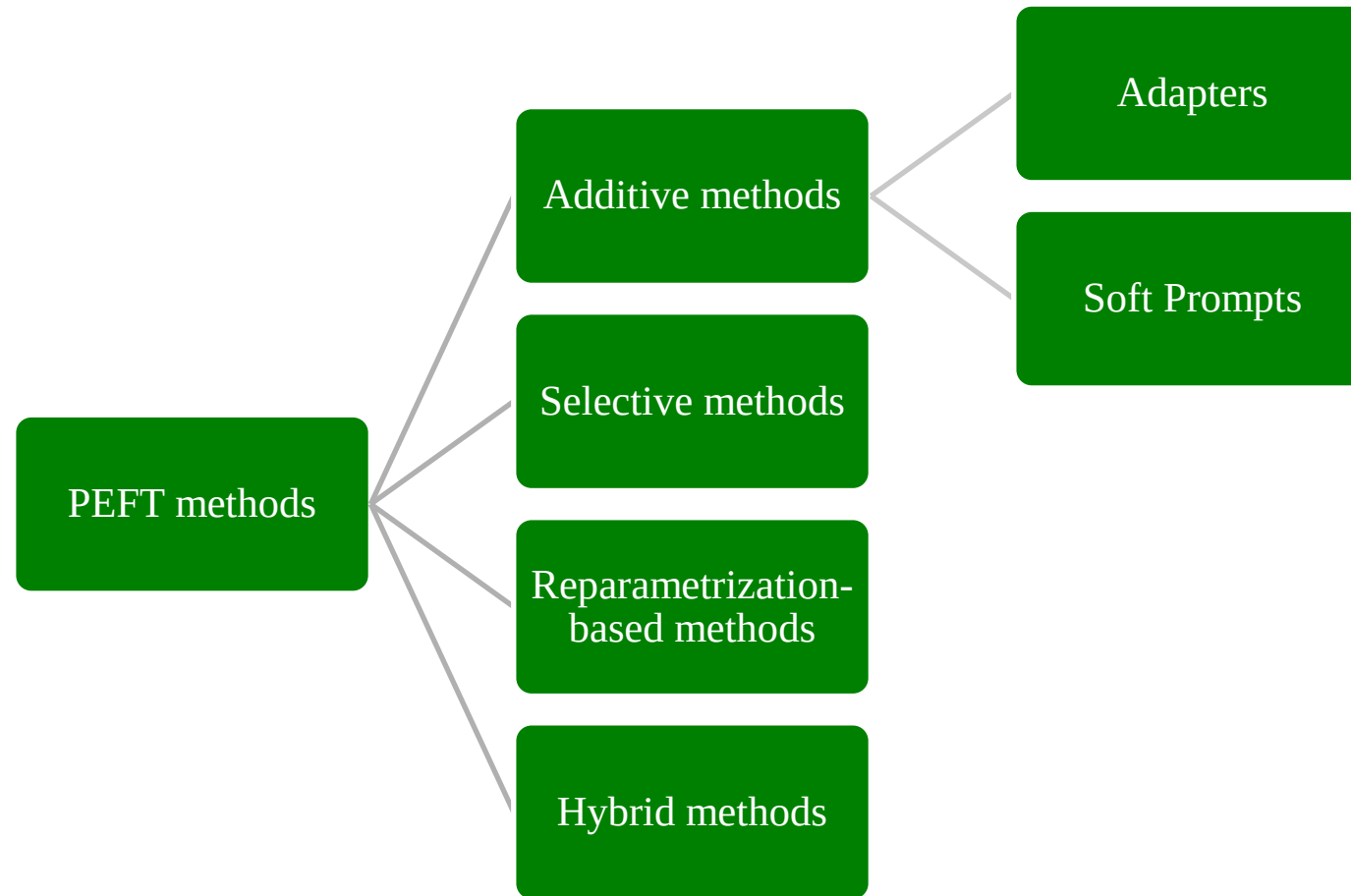
		FT									FT						
		125M	350M	1.3B	2.7B	6.7B	13B	30B			125M	350M	1.3B	2.7B	6.7B	13B	30B
ICL	125M	-0.00	0.01	0.02	0.03	0.12	0.14	0.09	ICL	125M	-0.00	0.00	0.02	0.01	0.10	0.11	0.07
	350M	-0.00	0.01	0.02	0.03	0.12	0.14	0.09		350M	-0.00	0.00	0.02	0.01	0.10	0.11	0.07
	1.3B	-0.00	0.01	0.02	0.03	0.12	0.14	0.09		1.3B	-0.01	-0.00	0.01	0.01	0.10	0.11	0.07
	2.7B	-0.00	0.01	0.02	0.03	0.12	0.14	0.09		2.7B	-0.01	-0.00	0.01	0.01	0.09	0.10	0.07
	6.7B	-0.00	0.01	0.02	0.03	0.12	0.14	0.09		6.7B	-0.01	-0.01	0.01	0.00	0.09	0.10	0.06
	13B	-0.04	-0.02	-0.01	-0.00	0.09	0.11	0.05		13B	-0.03	-0.03	-0.02	-0.02	0.07	0.08	0.04
	30B	-0.11	-0.09	-0.08	-0.08	0.02	0.03	-0.02		30B	-0.07	-0.07	-0.05	-0.06	0.03	0.04	0.00

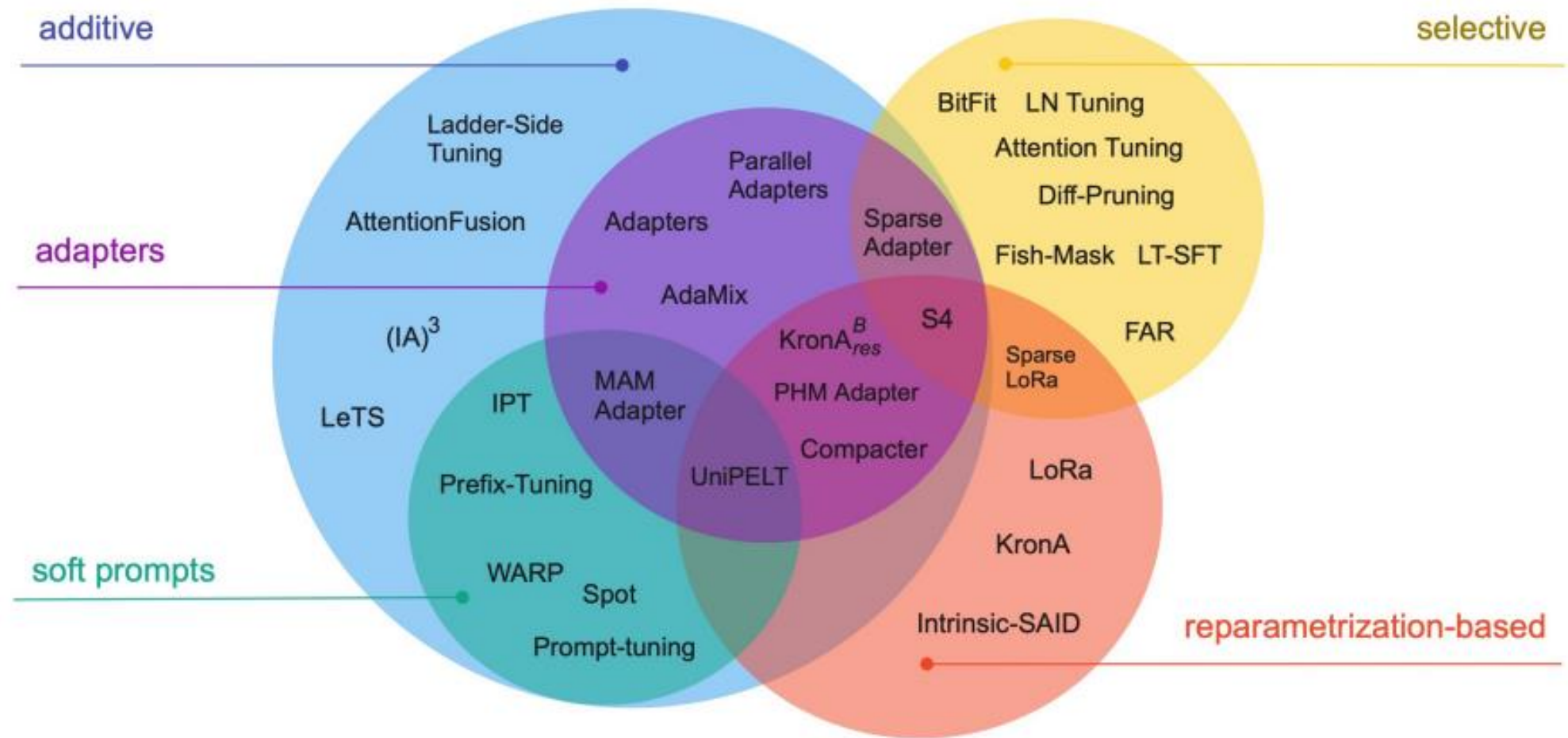
(a) RTE

(b) MNLI

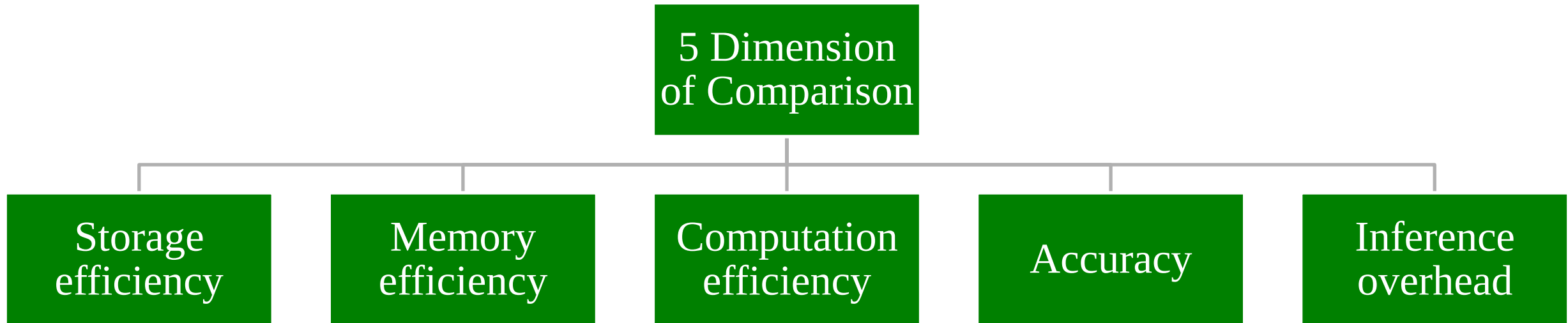
Table 1: Difference between average **out-of-domain performance** of ICL and FT on RTE (a) and MNLI (b) across model sizes. We use 16 examples and 10 random seeds for both approaches. For ICL, we use the `gpt-3` pattern. For FT, we use pattern-based fine-tuning (PBFT) and select checkpoints according to in-domain performance. We perform a Welch's t-test and color cells according to whether: **ICL performs significantly better than FT**, **FT performs significantly better than ICL**. For cells without color, there is no significant difference.

Taxonomy of PEFT: A birds-eye view





Comparing PEFT methods



Method	Type	Storage	Memory	Backprop	Inference overhead
Adapters (Houlsby et al., 2019)	A	yes	yes	no	Extra FFN
AdaMix (Wang et al., 2022)	A	yes	yes	no	Extra FFN
SparseAdapter (He et al., 2022b)	AS	yes	yes	no	Extra FFN
Cross-Attn tuning (Gheini et al., 2021)	S	yes	yes	no	No overhead
BitFit (Ben-Zaken et al., 2021)	S	yes	yes	no	No overhead
DiffPruning (Guo et al., 2020)	S	yes	no	no	No overhead
Fish-Mask (Sung et al., 2021)	S	yes	maybe ⁵	no	No overhead
LT-SFT (Ansell et al., 2022)	S	yes	maybe ⁵	no	No overhead
Prompt Tuning (Lester et al., 2021)	A	yes	yes	no	Extra input
Prefix-Tuning (Li and Liang, 2021)	A	yes	yes	no	Extra input
Spot (Vu et al., 2021)	A	yes	yes	no	Extra input
IPT (Qin et al., 2021)	A	yes	yes	no	Extra FFN and input
MAM Adapter (He et al., 2022a)	A	yes	yes	no	Extra FFN and input
Parallel Adapter (He et al., 2022a)	A	yes	yes	no	Extra FFN
Intrinsic SAID (Aghajanyan et al., 2020)	R	no	no	no	No overhead
LoRa (Hu et al., 2021)	R	yes	yes	no	No overhead
UniPELT (Mao et al., 2021)	AR	yes	yes	no	Extra FFN and input
Compacter (Karimi Mahabadi et al., 2021)	AR	yes	yes	no	Extra FFN
PHM Adapter (Karimi Mahabadi et al., 2021)	AR	yes	yes	no	Extra FFN
KronA (Edalati et al., 2022)	R	yes	yes	no	No overhead
KronA _{res} ^B (Edalati et al., 2022)	AR	yes	yes	no	Extra linear layer
(IA) ³ (Liu et al., 2022)	A	yes	yes	no	Extra gating
Attention Fusion (Cao et al., 2022)	A	yes	yes	yes	Extra decoder
LeTS (Fu et al., 2021)	A	yes	yes	yes	Extra FFN
Ladder Side-Tuning (Sung et al., 2022)	A	yes	yes	yes	Extra decoder
FAR (Vucetic et al., 2022)	S	yes	maybe ⁶	no	No overhead
S4-model (Chen et al., 2023)	ARS	yes	yes	no	Extra FFN and input

Overview of PEFT Approaches

Method	% Trainable parameters	% Changed parameters	Evaluated on		
			<1B	<20B	>20B
Adapters (Houlsby et al., 2019)	0.1 - 6	0.1 - 6	yes	yes	yes
AdaMix (Wang et al., 2022)	0.1 - 0.2	0.1 - 0.2	yes	no	no
SparseAdapter (He et al., 2022b)	2.0	2.0	yes	no	no
BitFit (Ben-Zaken et al., 2021)	0.05 - 0.1	0.05 - 0.1	yes	yes	yes
DiffPruning (Guo et al., 2020)	200	0.5	yes	no	no
Fish-Mask (Sung et al., 2021)	0.01 - 0.5	0.01 - 0.5	yes	yes	no
Prompt Tuning (Lester et al., 2021)	0.1	0.1	yes	yes	yes
Prefix-Tuning (Li and Liang, 2021)	0.1 - 4.0	0.1 - 4.0	yes	yes	yes
IPT (Qin et al., 2021)	56.0	56.0	yes	no	no
MAM Adapter (He et al., 2022a)	0.5	0.5	yes	no	no
Parallel Adapter (He et al., 2022a)	0.5	0.5	yes	no	no
Intrinsic SAID (Aghajanyan et al., 2020)	0.001 - 0.1	~0.1 or 100	yes	yes	no
LoRa (Hu et al., 2021)	0.01 - 0.5	~0.5 or ~30	yes	yes	yes
UniPELT (Mao et al., 2021)	1.0	1.0	yes	no	no
Compacter (Karimi Mahabadi et al., 2021)	0.05-0.07	~0.07 or ~0.1	yes	yes	no
PHM Adapter (Karimi Mahabadi et al., 2021)	0.2	~0.2 or ~1.0	yes	no	no
KronA (Edalati et al., 2022)	0.07	~0.07 or ~30.0	yes	no	no
KronA ^B _{res} (Edalati et al., 2022)	0.07	~0.07 or ~1.0	yes	no	no
(IA) ³ (Liu et al., 2022)	0.02	0.02	no	yes	no
Ladder Side-Tuning(Sung et al., 2022)	7.5	7.5	yes	yes	no
FAR (Vucetic et al., 2022)	6.6-26.4	6.6-26.4	yes	no	no
S4-model (Chen et al., 2023)	0.5	more than 0.5	yes	yes	no

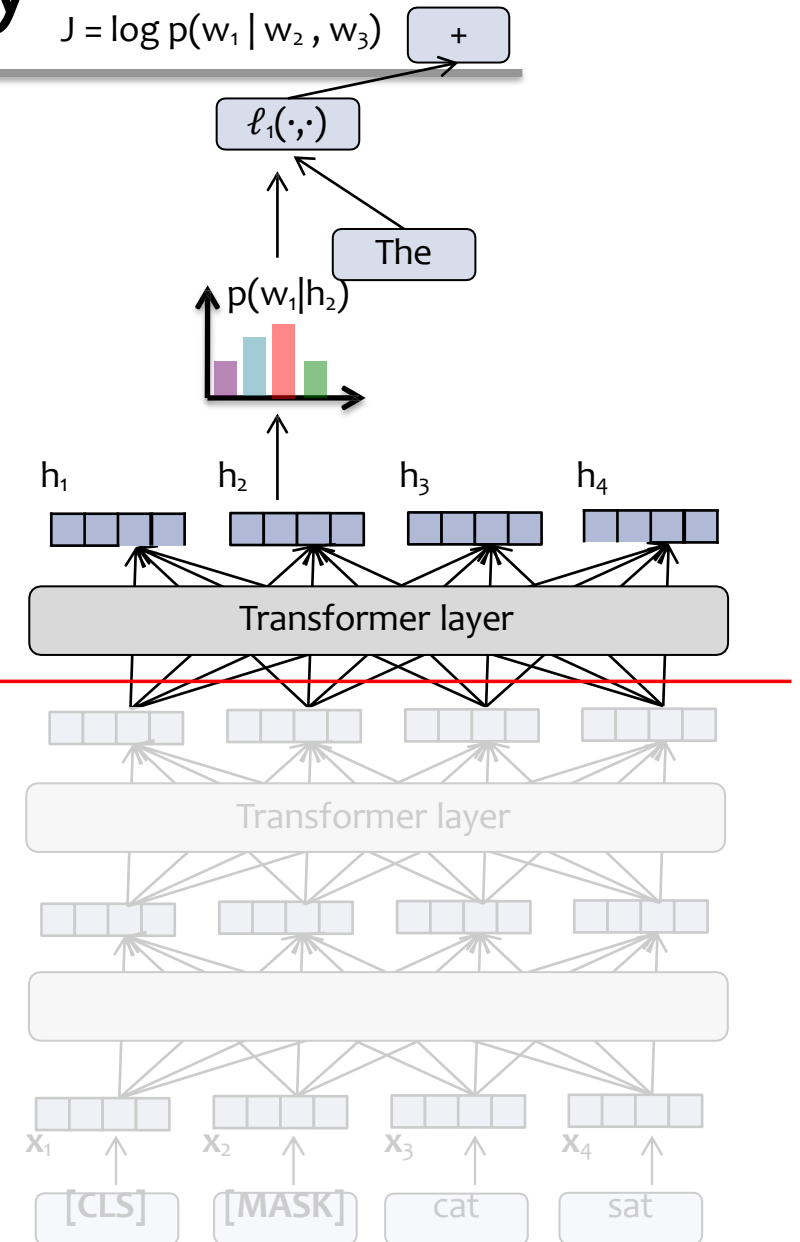
Parameter Efficient Fine-Tuning

- ❑ **Goal:** perform fine-tuning of fewer parameters, but achieve performance on a downstream task that is comparable to fine-tuning of all parameters
- ❑ **Various approaches:**
 - **Prefix Tuning:** for a Transformer LM, pretend as if there exist many tokens that came before your sequence and tune the keys/values corresponding to those tokens
 - **Subset:** Pick a subset of the parameters and fine-tune only those (e.g. only the top K layers of a K+L layer deep neural network)
 - **Adapters:** add additional layers that have few parameters and tune only the parameters of those layers, keeping all others fixed
 - **LoRA:** learn a small delta for each of the parameter matrices with the delta chosen to be low rank

Fine-Tuning the Top Layers Only

- ❑ Simple baseline for PEFT:
 - keep all parameters fixed except for the top K layers
 - gradients only need to flow through K layers instead of K+L total layers
 - reduced memory usage b/c we don't need to store the adjoints (gradient of the loss with respect to each parameter) for the full computation graph
- ❑ Can easily be applied to most deep neural networks

stop gradient here
s.t. error does not
backprop to lower
layers



Adapters

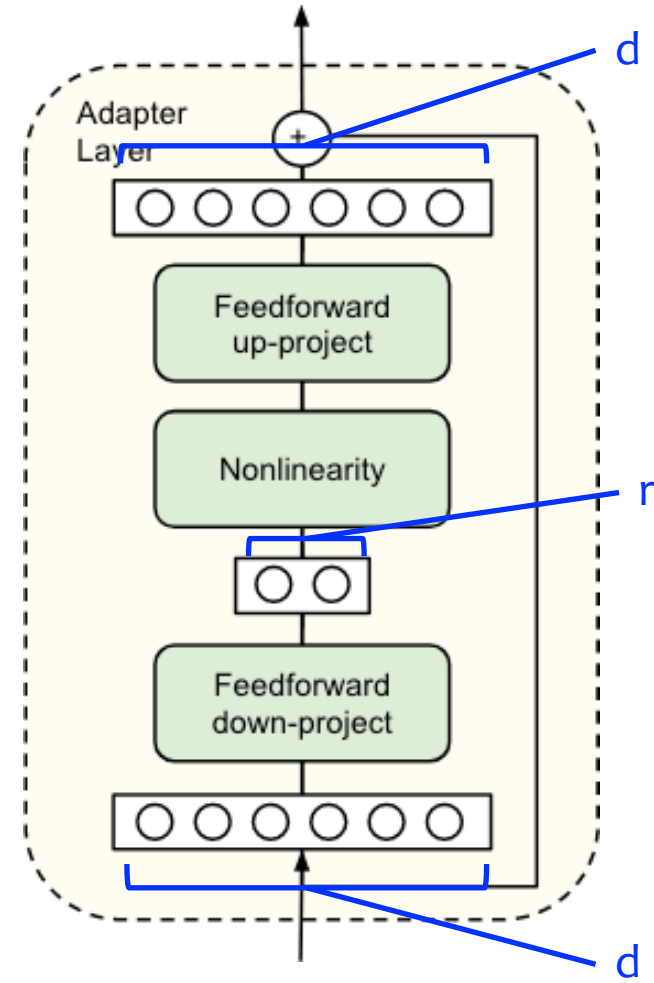
Overview of PEFT Approaches (Adapters)

- ❑ Add fully-connected networks after attention and FFN layers in Transformer.
- ❑ Possible to achieve performance competitive to full fine-tuning by tuning less than 4% of the total model parameters.

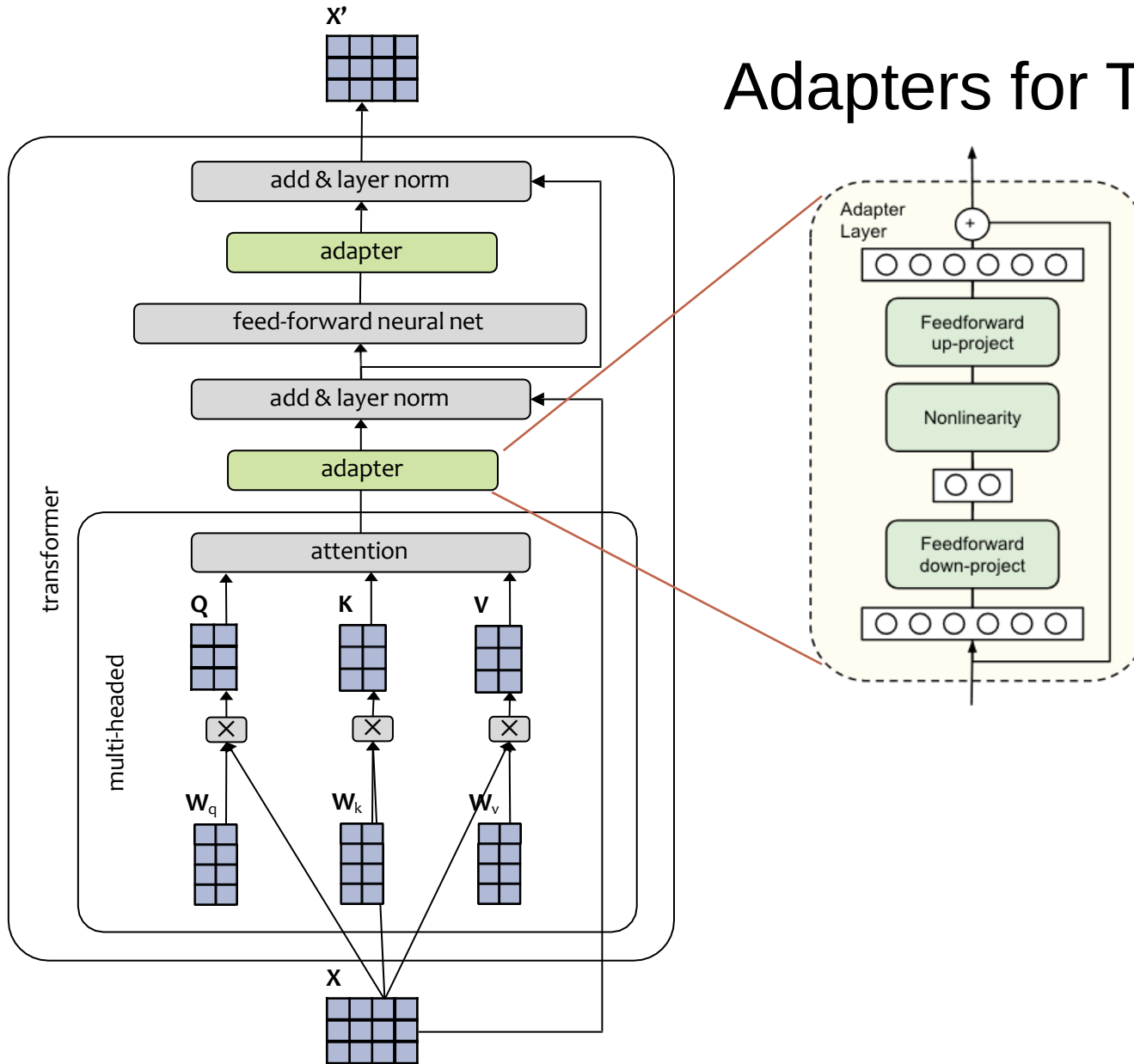
Type	A	% Train	0.1-6
Storage	Yes	% Change	0.1-6
Memory	Yes	<1B	Yes
Backprop	No	<20B	Yes
Inf. Overhead	Extra FFN	>20B	Yes

Adapters Module

- ❑ An adapter layer is simply a feed-forward neural network with one hidden layer, and a residual connection
- ❑ For input dimension, d , the adapter layer also has output dimension d , but bottlenecks to a lower dimension m in the middle



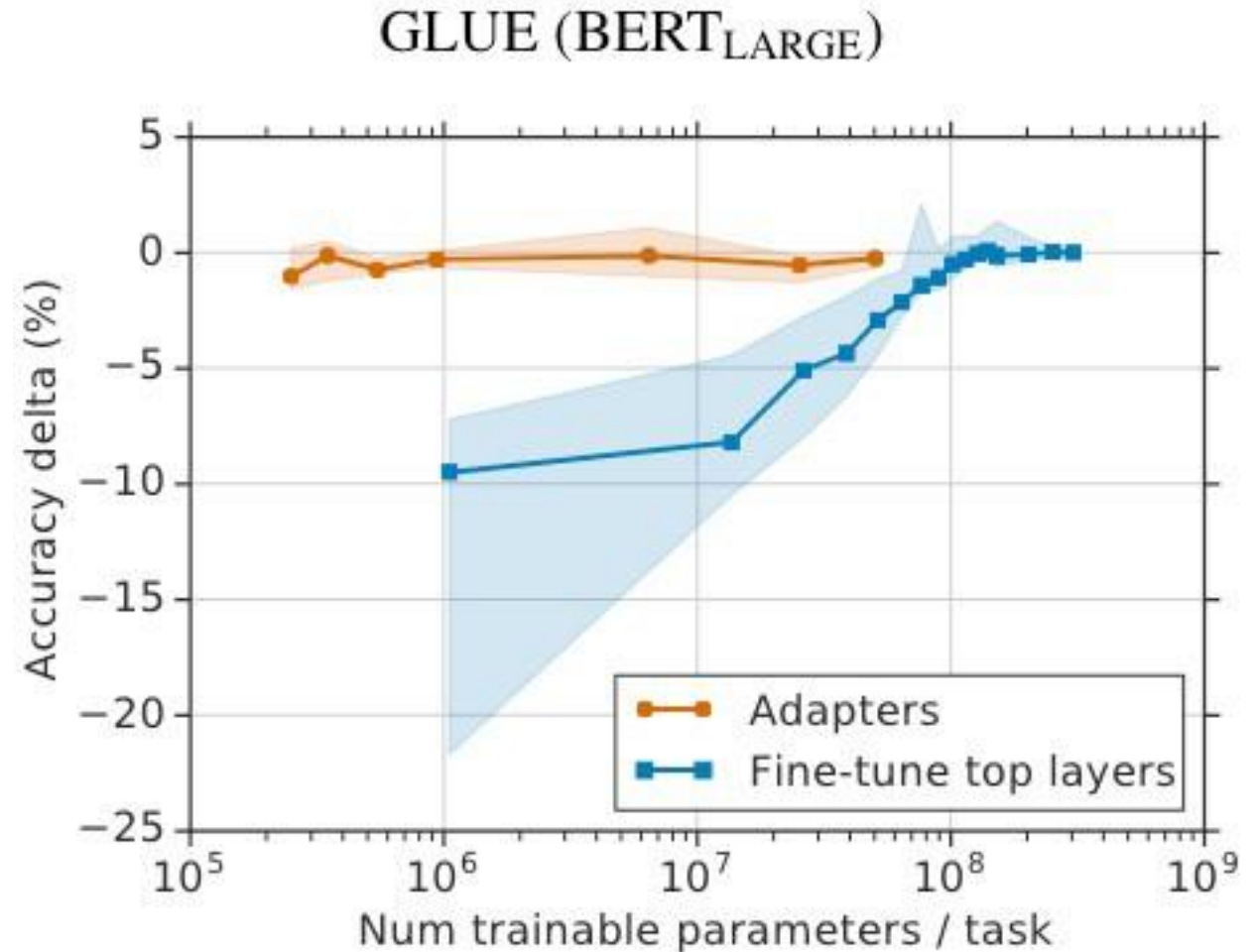
Adapters for Transformer



- In practice, m is chosen s.t. $m \ll d$ and the adapter layers contain only 0.5% – 8% of the total parameters
- When added to a deep neural network (e.g. Transformer) all the other parameters of the pretrained model are kept fixed, and only the adapter layer parameters are fine-tuned

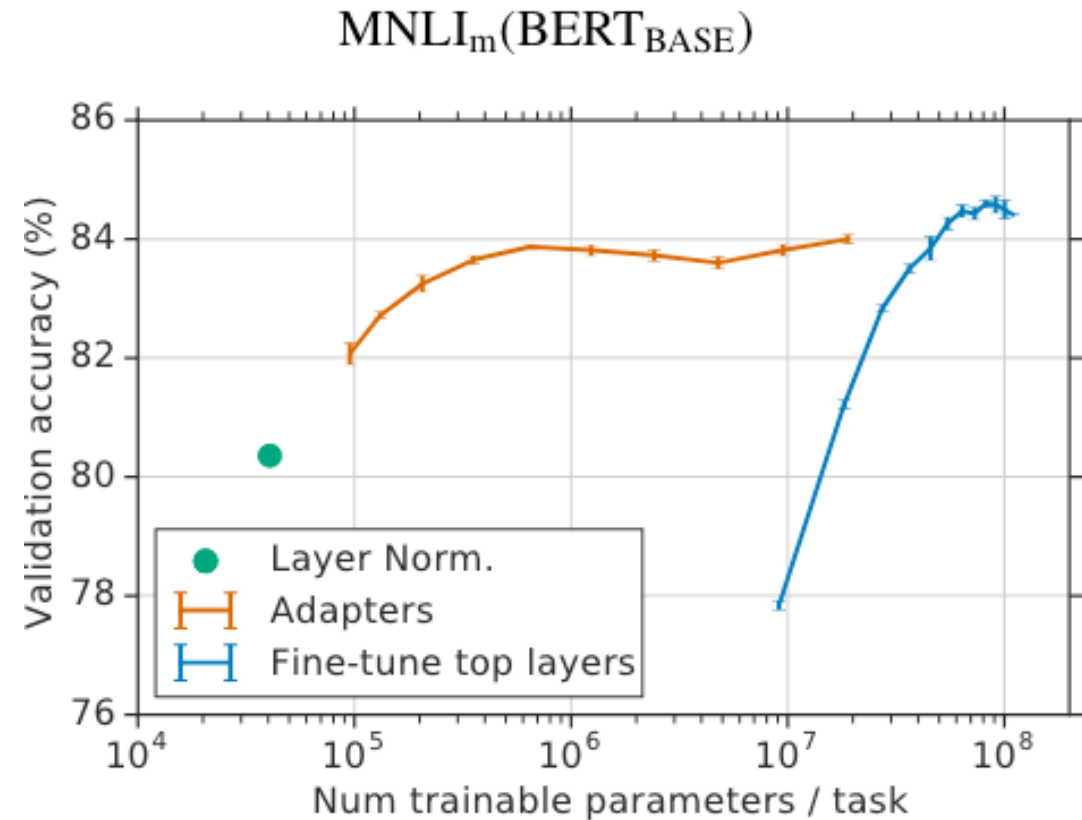
Adapter Results

- ❑ **Pretrained Model:** BERT- Large
- ❑ **Baseline Method:** fine- tune only the top K layers of BERT-Large
- ❑ Adapters achieve nearly the performance (i.e. 0% delta) of full fine-tuning but with substantially fewer parameters
- ❑ Sometimes adapters even outperform full fine-tuning



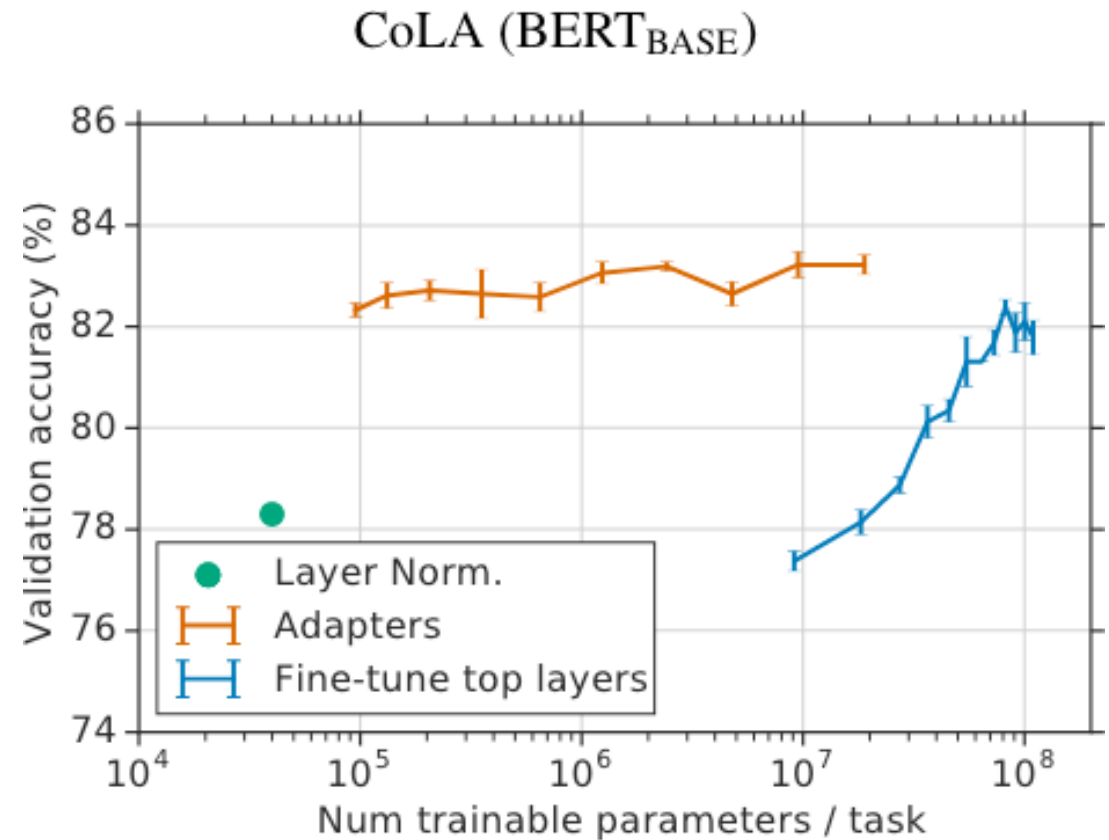
Adapter Results

- ❑ **Pretrained Model:** BERT- Large
- ❑ **Baseline Method:** fine- tune only the top K layers of BERT-Large
- ❑ Adapters achieve nearly the performance (i.e. 0% delta) of full fine-tuning but with substantially fewer parameters
- ❑ Sometimes adapters even outperform full fine-tuning



Adapter Results

- ❑ **Pretrained Model:** BERT- Large
- ❑ **Baseline Method:** fine- tune only the top K layers of BERT-Large
- ❑ Adapters achieve nearly the performance (i.e. 0% delta) of full fine-tuning but with substantially fewer parameters
- ❑ Sometimes adapters even outperform full fine-tuning



Low-Rank Adaptation (LoRA)

How large are LLMs?

Recall...

- Comparison of some recent **large language models** (LLMs)

Model	Creators	Year of release	Training Data (# tokens)	Model Size (# parameters)
GPT-2	OpenAI	2019	~10 billion (40Gb)	1.5 billion
GPT-3 (cf. ChatGPT)	OpenAI	2020	300 billion	175 billion
PaLM	Google	2022	780 billion	540 billion
Chinchilla	DeepMind	2022	1.4 trillion	70 billion
LaMDA (cf. Bard)	Google	2022	1.56 trillion	137 billion
LLaMA	Meta	2023	1.4 trillion	65 billion
LLaMA-2	Meta	2023	2 trillion	70 billion
GPT-4	OpenAI	2023	?	?

Why does efficiency matter?

Recall...

□ Case Study: GPT-3

- # of training tokens = 500 billion
- # of parameters = 175 billion
- # of cycles = 50 petaflop/s-days (each of which are 8.64×10^{19} flops)

Dataset	Quantity (tokens)	Weight in training mix	Epochs elapsed when training for 300B tokens
Common Crawl (filtered)	410 billion	60%	0.44
WebText2	19 billion	22%	2.9
Books1	12 billion	8%	1.9
Books2	55 billion	8%	0.43
Wikipedia	3 billion	3%	3.4

Table 2.2: Datasets used to train GPT-3. “Weight in training mix” refers to the fraction of examples during training that are drawn from a given dataset, which we intentionally do not make proportional to the size of the dataset. As a result, when we train for 300 billion tokens, some datasets are seen up to 3.4 times during training while other datasets are seen less than once.

Model Name	n_{params}	n_{layers}	d_{model}	n_{heads}	d_{head}	Batch Size	Learning Rate
GPT-3 Small	125M	12	768	12	64	0.5M	6.0×10^{-4}
GPT-3 Medium	350M	24	1024	16	64	0.5M	3.0×10^{-4}
GPT-3 Large	760M	24	1536	16	96	0.5M	2.5×10^{-4}
GPT-3 XL	1.3B	24	2048	24	128	1M	2.0×10^{-4}
GPT-3 2.7B	2.7B	32	2560	32	80	1M	1.6×10^{-4}
GPT-3 6.7B	6.7B	32	4096	32	128	2M	1.2×10^{-4}
GPT-3 13B	13.0B	40	5140	40	128	2M	1.0×10^{-4}
GPT-3 175B or “GPT-3”	175.0B	96	12288	96	128	3.2M	0.6×10^{-4}

Table 2.1: Sizes, architectures, and learning hyper-parameters (batch size in tokens and learning rate) of the models which we trained. All models were trained for a total of 300 billion tokens.

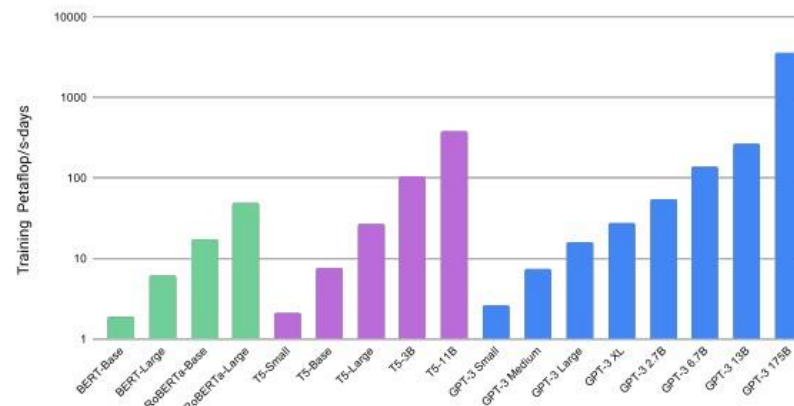


Figure 2.2: Total compute used during training. Based on the analysis in Scaling Laws For Neural Language Models [KMH⁺20] we train much larger models on many fewer tokens than is typical. As a consequence, although GPT-3 3B is almost 10x larger than RoBERTa-Large (355M params), both models took roughly 50 petaflop/s-days of compute during pre-training. Methodology for these calculations can be found in Appendix D.

Fine-Tuning LLMs without Regularization

Method	MNLI-m (Val. Acc./%)	RTE (Val. Acc./%)
GPT-3 Few-Shot	40.6	69.0
GPT-3 Fine-Tuned	89.5	85.4

Question:

Why don't LLMs overfit when we fine-tune them without regularization?

Hypothesis:

They are intrinsically low dimensional

Intrinsic Dimensionality

- ❑ Using similar techniques, Aghajanyan et al. (2020) measure the intrinsic dimension of LLMs
- ❑ Empirical results suggest that pre-training finds parameters that have low intrinsic dimensionality
- ❑ Number of parameters:
 - BERT-Base: 110 million
 - BERT-Large: 345 million

Model	SAID		DID	
	MRPC	QQP	MRPC	QQP
BERT-Base	1608	8030	1861	9295
BERT-Large	1037	1200	2493	1389
RoBERTa-Base	896	896	1000	1389
RoBERTa-Large	207	774	322	774

Table 1: Estimated d_{90} intrinsic dimension for a set of sentence prediction tasks and common pre-trained models. We present both the *SAID* and *DID* methods.

LoRA

❑ Motivation #1:

- “We take inspiration from Li et al. (2018a); Aghajanyan et al. (2020) which show that the learned over-parametrized models in fact reside on a low intrinsic dimension.”

❑ Motivation #2:

- Directly optimizing the prompt, as in prefix tuning, leads to non-monotonic changes in performance as the number of parameters increases (we want more parameters to mean better performance!)

❑ Motivation #3:

- Adapters and related methods introduce inference latency at test time that is non-trivial!

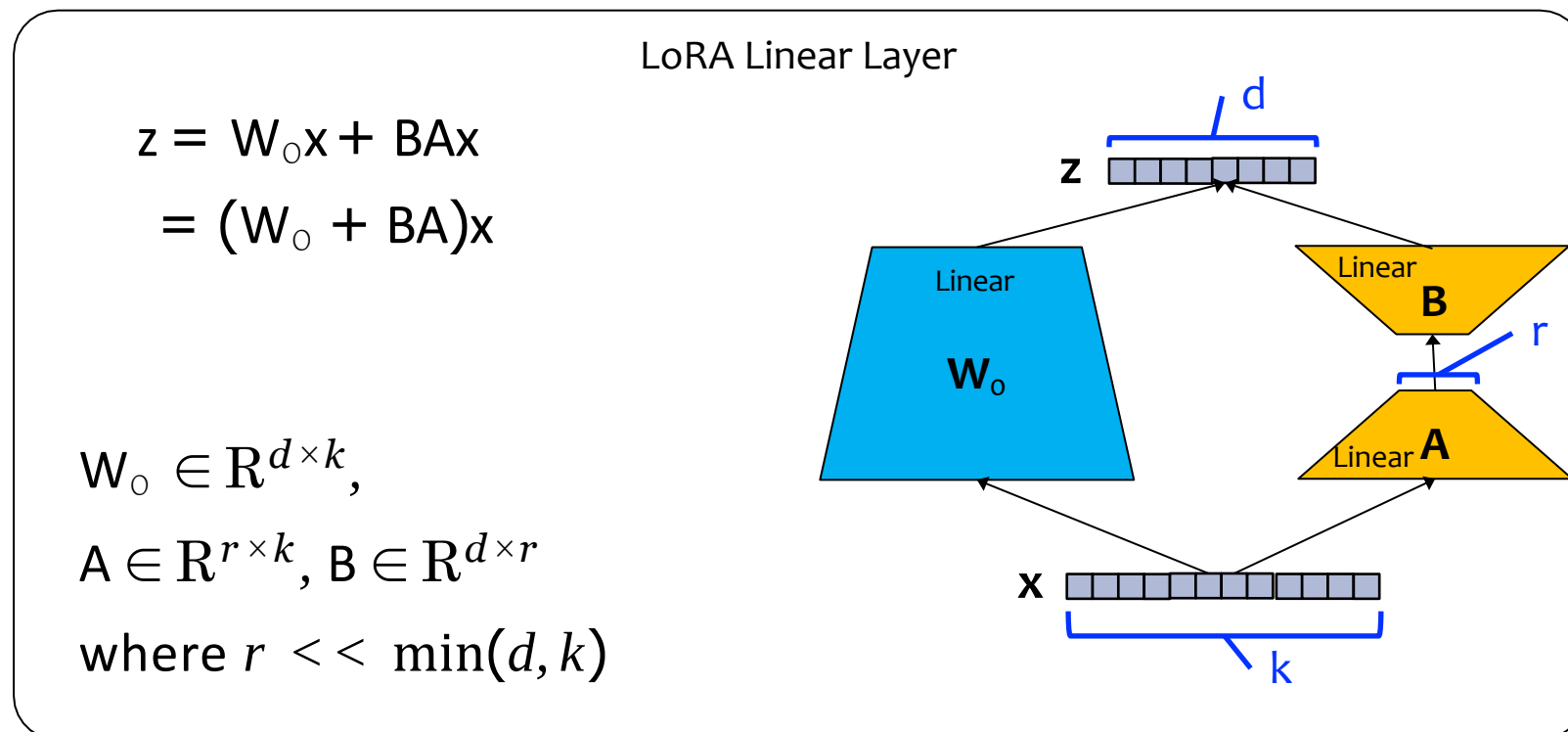
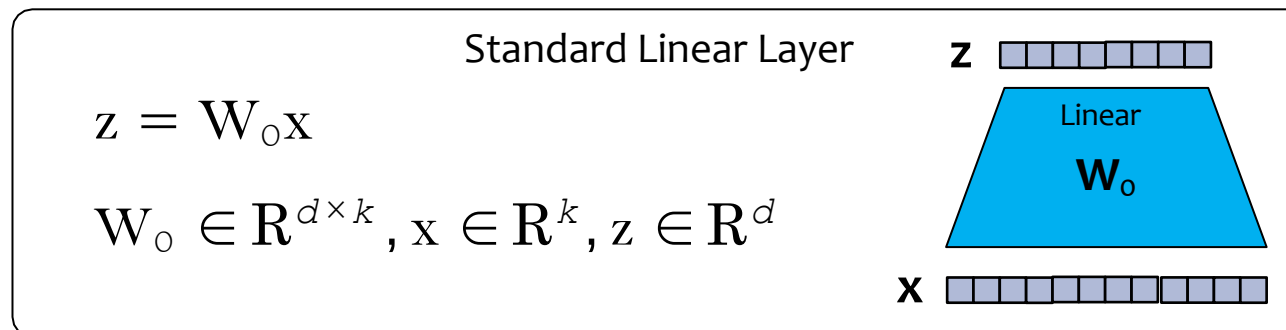
Batch Size	32	16	1
Sequence Length	512	256	128
$ \Theta $	0.5M	11M	11M
Fine-Tune/LoRA	1449.4±0.8	338.0±0.6	19.8±2.7
Adapter ^L	1482.0±1.0 (+2.2%)	354.8±0.5 (+5.0%)	23.9±2.1 (+20.7%)
Adapter ^H	1492.2±1.0 (+3.0%)	366.3±0.5 (+8.4%)	25.8±2.2 (+30.3%)

Table 1: Inference latency of a single forward pass in GPT-2 medium measured in milliseconds, averaged over 100 trials. We use an NVIDIA Quadro RTX8000. “ $|\Theta|$ ” denotes the number of trainable parameters in adapter layers. Adapter^L and Adapter^H are two variants of adapter tuning, which we describe in [Section 5](#). The inference latency introduced by adapter layers can be significant in an online, short-sequence-length scenario. See the full study in [Appendix B](#).

LoRA

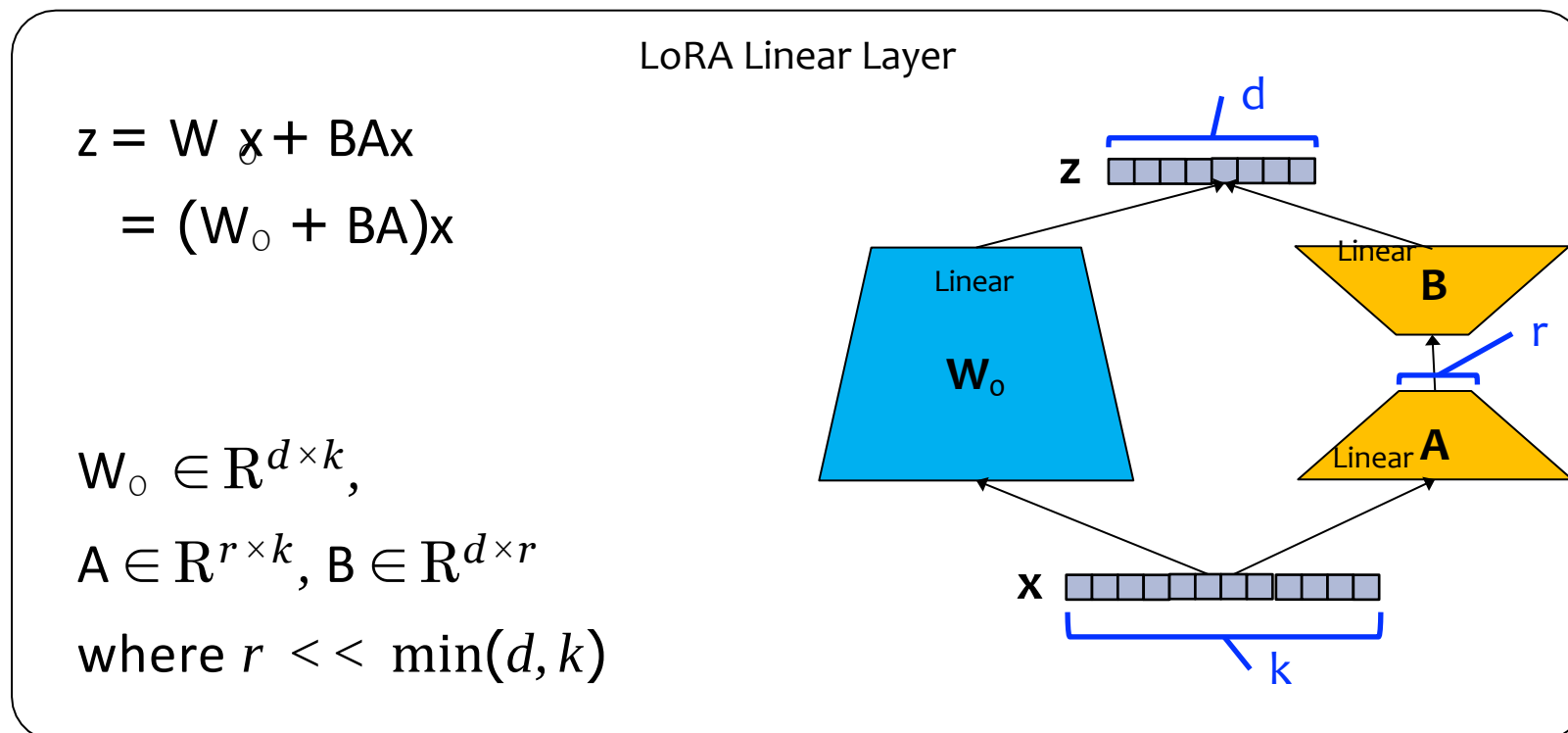
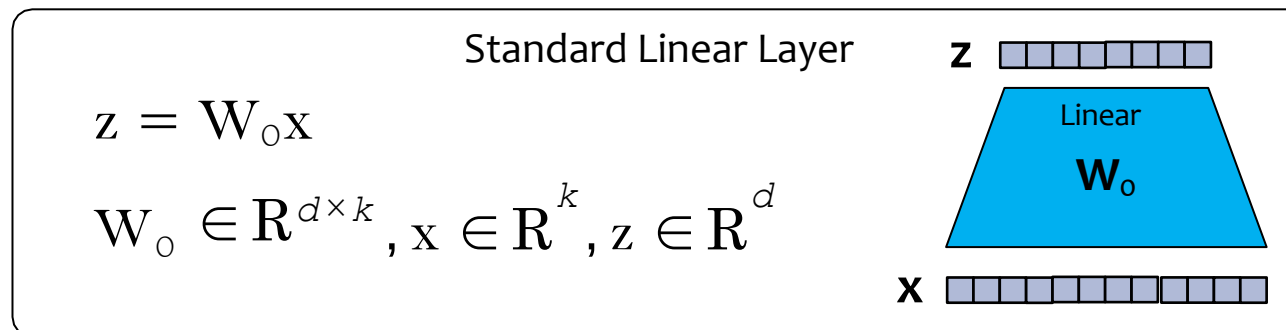
- Keep the original pretrained parameters W_0 fixed during fine-tuning
- Learn an additive modification to those parameters ΔW
- Define ΔW via a low rank decomposition:
 $\Delta W = BA$

where BA has rank r , which is much less than the input dimension k or the output dimension d



LoRA

- Initialization
- We initialize the trainable parameters:
$$A_{ij} \sim N(0, \sigma^2), \forall i, j$$
$$B = 0$$
- This ensures that, at the start of fine tuning, the parameters have their pretrained values:
$$\Delta W = BA = 0$$
 - $W_0 + BA = W_0$



Overview of PEFT Approaches (LoRA)

Type	R	% Train	0.01-0.5
Storage	Yes	% Change	0.5-30
Memory	Yes	<1B	Yes
Backprop	No	<20B	Yes
Inf. Overhead	No	>20B	Yes

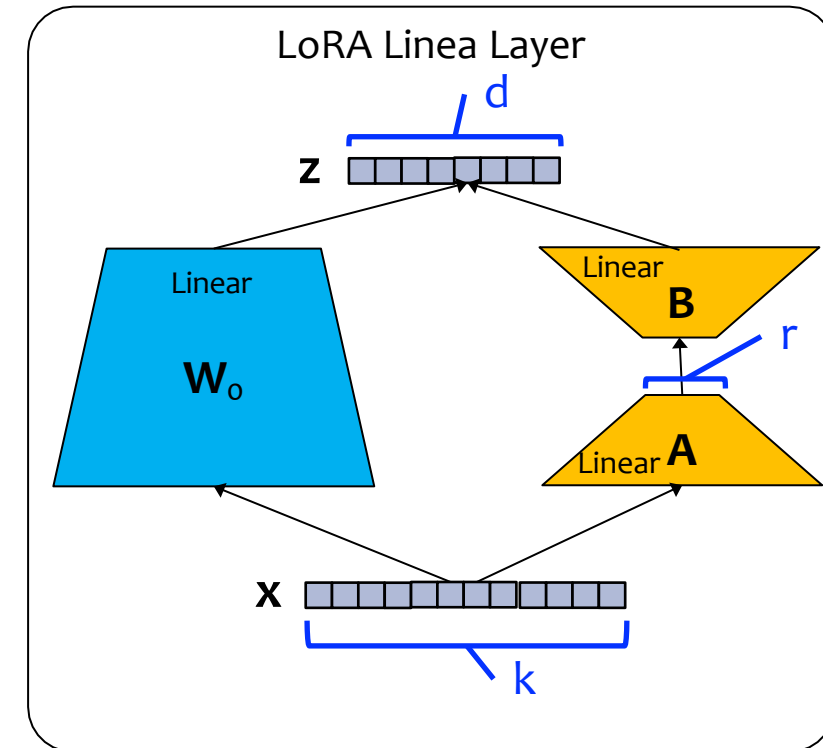
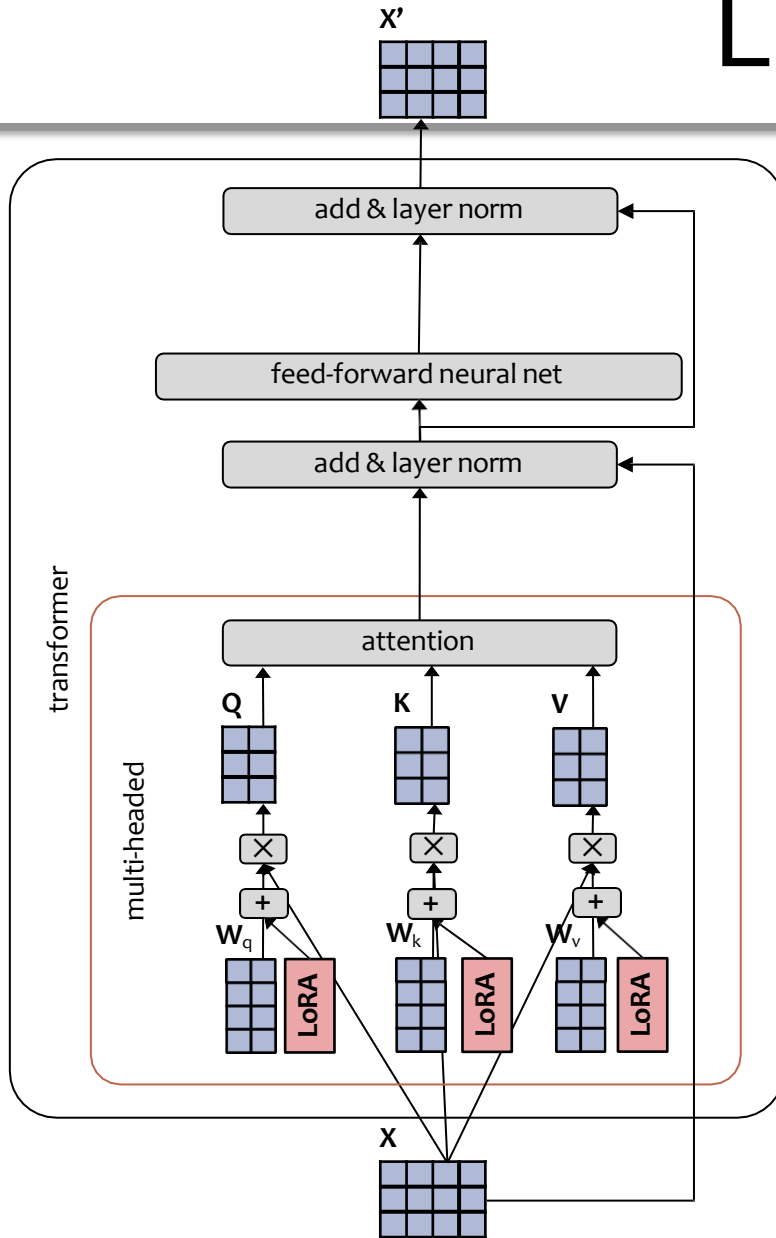
LoRA for Transformer

- LoRA linear layers could replace every linear layer in the Transformer layer
- But the original paper only applies LoRA to the attention weights

$$Q = \text{LoRALinear}(X; W_q, A_q, B_q)$$

$$K = \text{LoRALinear}(X; W_k, A_k, B_k)$$

$$V = \text{LoRALinear}(X; W_v, A_v, B_v)$$



LoRA Results

Takeaways

- Applied to GPT-3, LoRA achieves performance almost as good as full fine-tuning, but with far fewer parameters
- On some tasks it even outperforms full fine-tuning
- For some datasets a rank of $r=1$ is sufficient

